

Deep Learning

Lecture 7

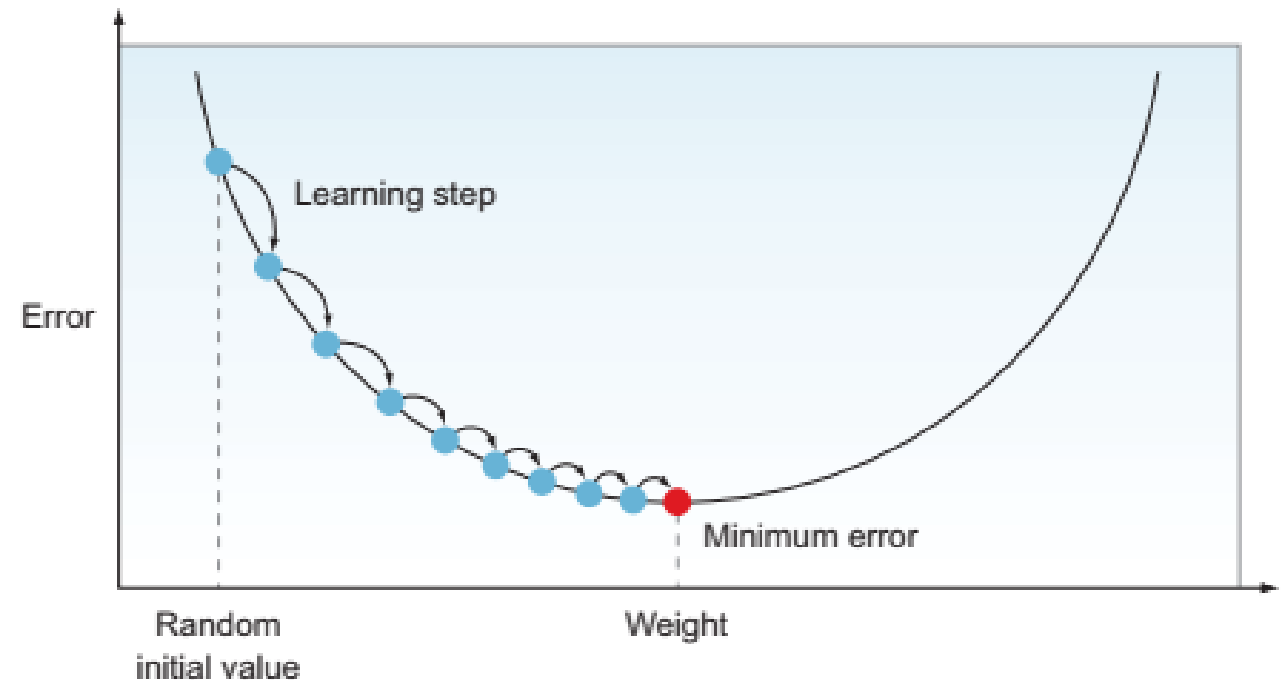
Learning and optimization

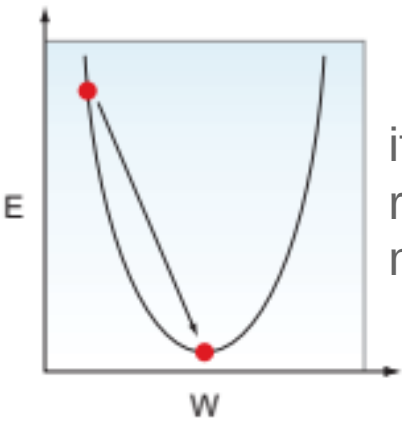
Learning rate and decay schedule

The learning rate is the single most important hyperparameter, and one should always make sure that it has been tuned. If there is only time to optimize one hyperparameter, then this is the hyperparameter that is worth tuning .

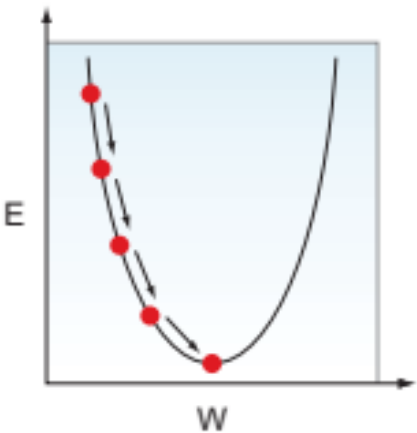
The GD optimizer searches for the optimal values of weights that yield the lowest error possible. When setting up our optimizer, we need to define the step size that it takes when it descends the error mountain. This step size is the learning rate.

The GD calculates the gradient to find the direction

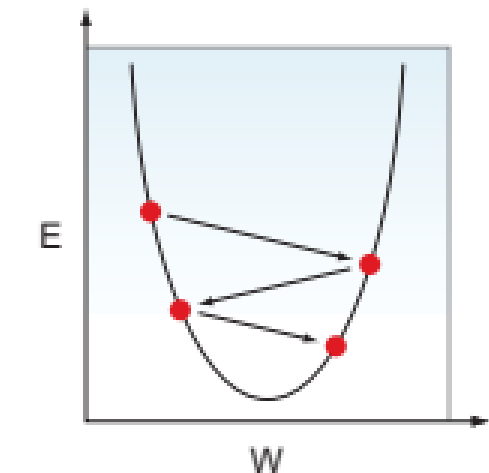




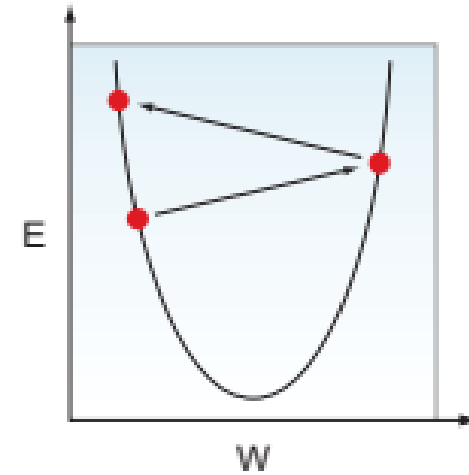
if we make a *miraculously* correct choice of the learning rate value, we land on the best weight value that minimizes the error in only one step



A learning rate *smaller* than the ideal lr value: the model takes smaller steps down the error curve.



A learning rate *larger* than the ideal lr value: the optimizer overshoot the optimal weight value.



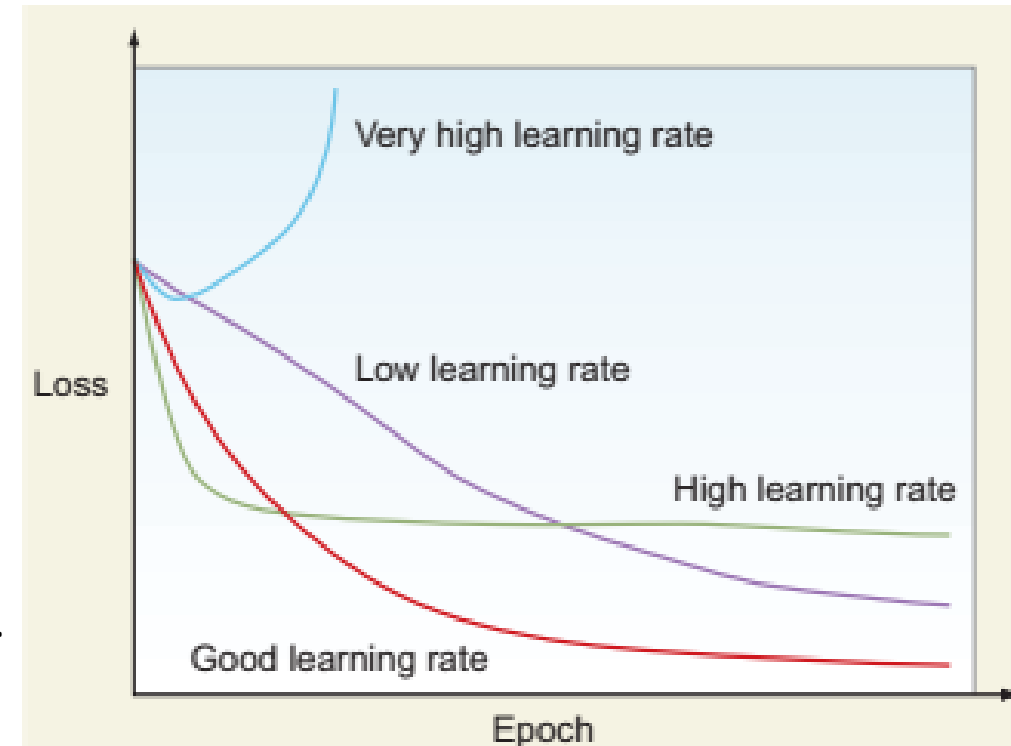
A learning rate much larger than the ideal lr value: the optimizer gets farther from the min error.

Too-high vs. too-low learning rate

- Setting the learning rate high or low is a trade-off between the optimizer speed versus performance.
- Too-low lr requires many epochs to converge, often too many. Theoretically, if the learning rate is too small, the algorithm is guaranteed to eventually converge if kept running for infinity time.
- Too-high lr might get us to a lower error value faster because we take bigger steps down the error curve, but there is a better chance that the algorithm will oscillate and diverge away from the minimum value. So, ideally, we want to pick the lr that is just right (optimal): it swiftly reaches the minimum point without being so big that it might diverge.

When plotting the loss value against the number of training iterations (epochs), you will notice the following:

- Much smaller lr—The loss keeps decreasing but needs a lot more time to converge.
- Larger lr—The loss achieves a better value than what we started with, but is still far from optimal.
- Much larger lr—The loss might initially decrease, but it starts to increase as the weight values get farther and farther away from the optimal values.
- Good lr—The loss decreases consistently until it reaches the minimum possible value.



The way to **debug** this is to look at the validation loss values in the training **verbose**:

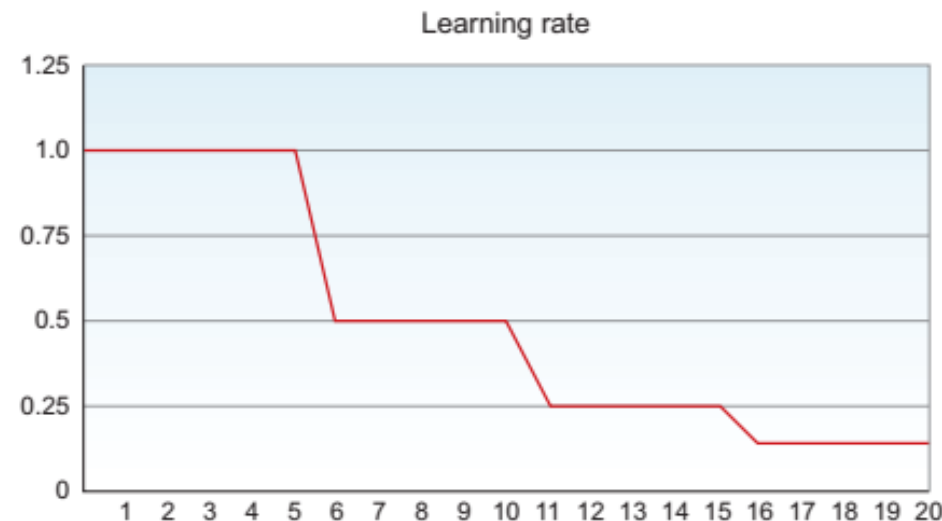
- If val_loss decreases after each step, that's good. Keep training until it stops improving.
- If training is complete and val_loss is **still decreasing**, then maybe the learning rate was so small that it didn't converge yet. In this case, you can do one of two things:
 - Train again with the same learning rate but with **more training iterations** (epochs) to give the optimizer more time to converge.
 - **Increase the lr** value a little and train again
- If val_loss starts to increase or oscillate up and down, then the learning rate **is too high** and you need to decrease its value.

Epoch 1, Training Error: 5.4353, Validation Error: 5.6394

Epoch 2, Training Error: 5.1364, Validation Error: 5.2216

Epoch 3, Training Error: 4.7343, Validation Error: 4.8337

- Another way to go about tuning your learning rate is to set a **learning rate decay**: a method by which the learning rate changes *during* training.
- It often performs better than a static value, and drastically reduces the time required to get optimal results.
- The decay rate tells our network to **automatically decrease the lr** throughout the training process.
- For example, we can decrease the lr by a constant value of (x) for each (n) number of steps. This way, we can start with the higher value to take bigger steps toward the minimum, and then gradually decrease the learning rate every (n) epochs to avoid overshooting the ideal lr.
- One way to accomplish this is by reducing the learning rate linearly (*linear decay*).



- Another way is to decrease the lr exponentially (*exponential decay*).
- For example, you can **multiply it by 0.1 every eight epochs**. Clearly, the network will converge a lot slower than with linear decay, but it will eventually converge.

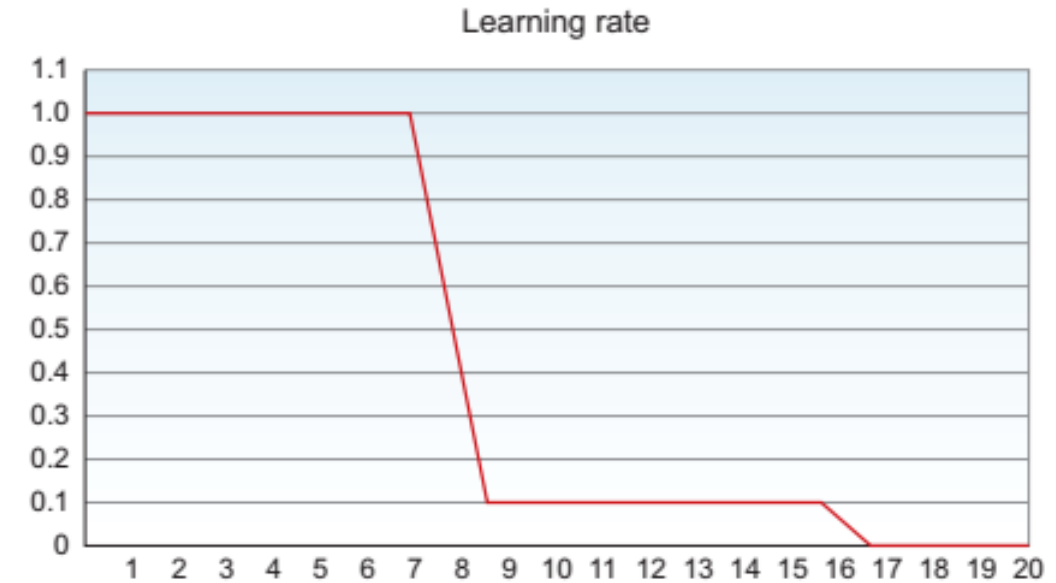
Other clever learning algorithms have an adaptive learning rate (*adaptive learning*).

These algorithms use a heuristic approach that automatically updates the lr when the training stops.

This means **not only decreasing** the lr when needed, but **also increasing** it when improvements are too slow (too-small lr).

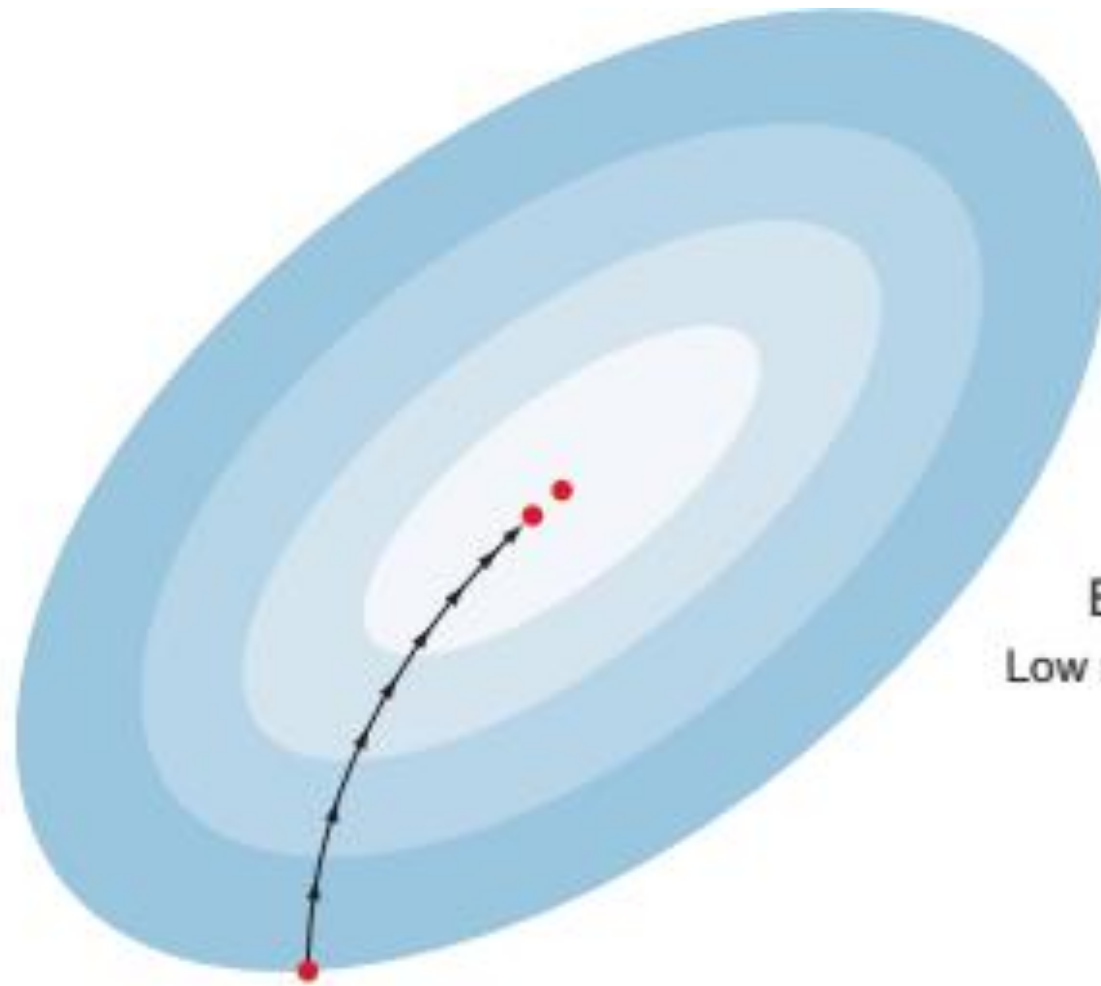
Adaptive learning usually works better than other learning rate-setting strategies.

Adam and **Adagrad** are examples of adaptive learning optimizers, we will study them in next three or four slide .



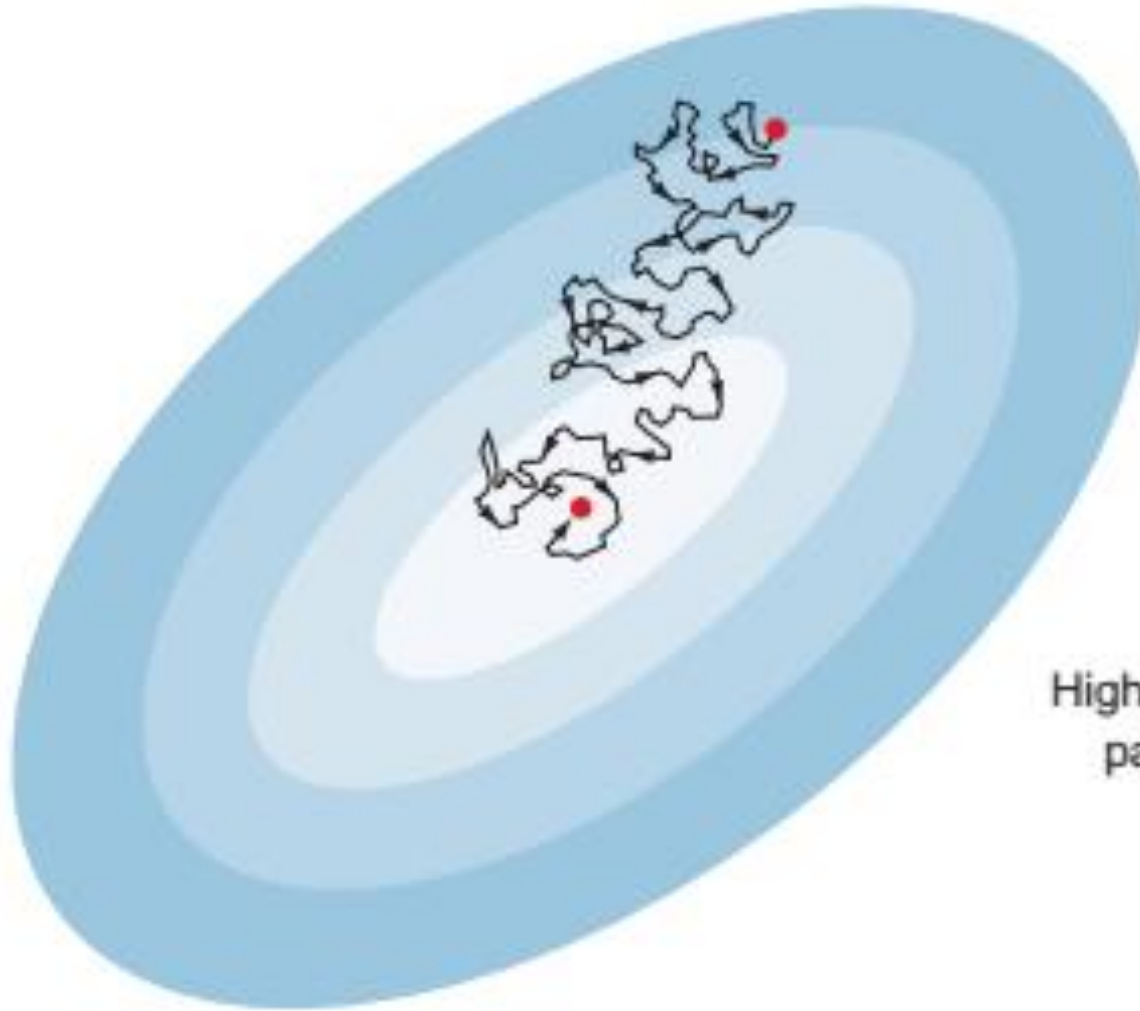
Mini-batch size

- Mini-batch size is another hyperparameter that you need to set and tune in the optimizer algorithm. The `batch_size` hyperparameter has **a big effect on resource requirements of the training process and speed**.
- ***Batch gradient descent (BGD)***—We feed the entire dataset to the network all at once, apply the feedforward process, calculate the error, calculate the gradient, and backpropagate to update the weights.
- The optimizer calculates the gradient by looking at the error generated after it sees *all* the training data, and the weights are updated *only once* after each epoch. So, in this case, the mini-batch size equals the entire training dataset.
- The main **advantage** of BGD is that it has relatively low noise and bigger steps toward the minimum.
- The main **disadvantage** is that it can take too long to process the entire training dataset at each step, especially when training on big data. BGD also requires a huge amount of memory for training large datasets, which might not be available. BGD might be a good option if you are training on a small dataset



Batch gradient descent (BGD)
Low noise on its path to the minimum error

- **Stochastic gradient descent (SGD)**—Also called *online learning*. We feed the network a single instance of the training data at a time and use this one instance to do the forward pass, calculate error, calculate the gradient, and backpropagate to update the weights. In SGD, the weights are updated after it sees each single instance (as opposed to processing the entire dataset before each step for BGD).
- SGD can be extremely noisy as it oscillates on its way to the **global minimum** because it takes a step down after each single instance, which could sometimes be in the wrong direction.
- This noise can be reduced by using a smaller learning rate, so, on average, it takes you in a good direction and almost always performs better than BGD.
- With SGD you get to make progress quickly and usually reach very close to the global minimum.
- The main disadvantage is that by calculating the GD for one instance at a time, **you lose the speed gain that comes with matrix multiplication in the training calculations.**

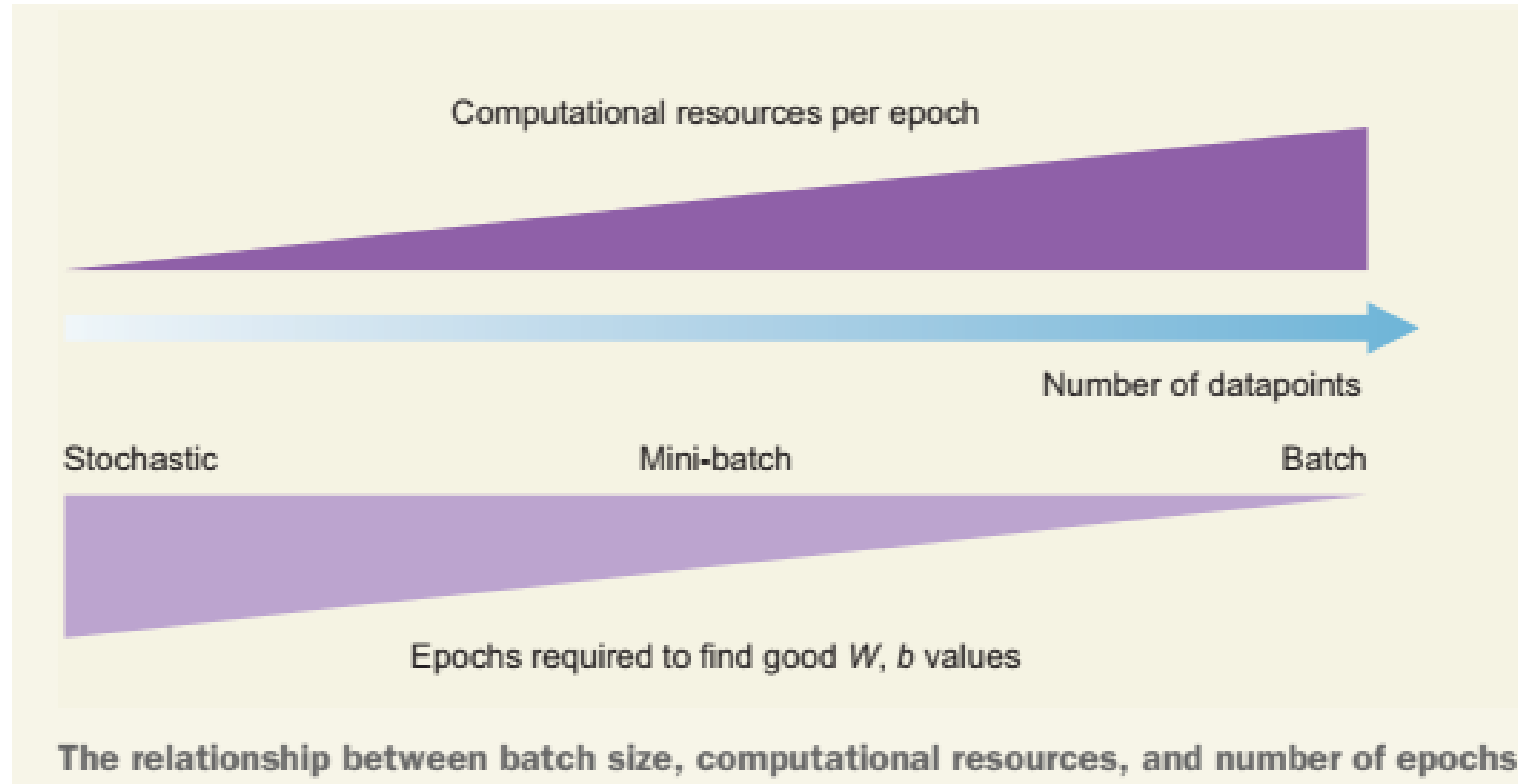


Stochastic (GD)

High noise and oscillates on its path to the minimum error

- **Mini-batch gradient descent (MB-GD)**—A compromise between batch and stochastic GD. Instead of computing the gradient from one sample (SGD) or all training samples (BGD), we divide the training sample into mini-batches to compute the gradient from. This way, we can take advantage of matrix multiplication for faster training and start making progress instead of having to wait to train the entire training set.
- **Guidelines for choosing mini-batch size**
First, if you have a small dataset (around less than 2,000), you might be better off using BGD. You can train the entire dataset quite fast.
- For large datasets, you can use a scale of mini-batch size values. A typical starting value for the mini-batch is 64 or 128. You can then tune it up and down on this scale: 32, 64, 128, 256, 512, 1024, and keep doubling it as needed to speed up training. But make sure that your mini-batch size fits in your CPU/GPU memory.
- Mini-batch sizes of 1024 and larger are possible but quite rare. A larger mini-batch size allows a computational boost that uses matrix multiplication in the training calculations. But that comes at the expense of needing more memory for the training process and generally more computational resources.

- figure shows the relationship between batch size, computational resources, and number of epochs needed for neural network training

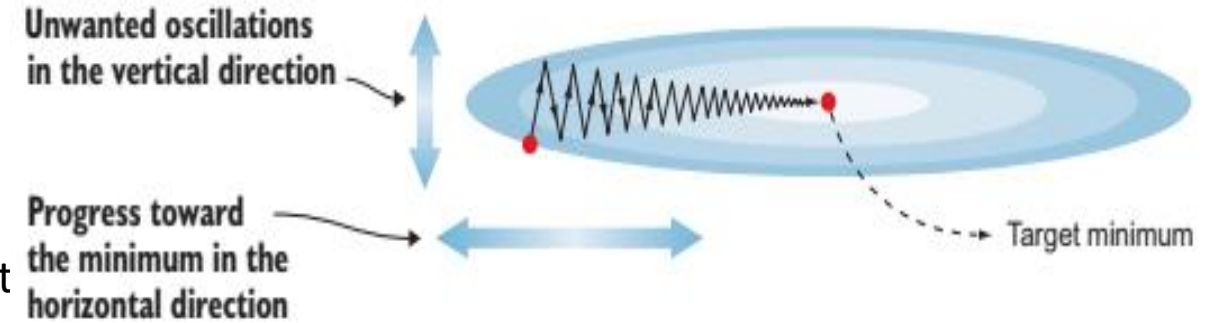


Optimization algorithms

GD with Momentum

Recall that SGD ends up with some oscillations in the vertical direction toward the minimum error.

These oscillations slow down the convergence process and make it harder to use larger learning rates, which could result in your algorithm overshooting and diverging.



- To reduce these oscillations, a technique called **momentum** was invented that lets the GD navigate along relevant directions and softens the oscillation in irrelevant directions. In other words, it makes learning slower in the vertical-direction oscillations and faster in the horizontal-direction progress, which will help the optimizer reach the target minimum much faster.

How the math works in momentum

The math here is really simple and straightforward. The momentum is built by adding a velocity term to the equation that updates the weight:

$$w_{\text{new}} = w_{\text{old}} - \alpha \frac{dE}{dw_i} \quad \leftarrow \text{Original update rule}$$

$$w_{\text{new}} = w_{\text{old}} - \text{learning rate} \times \text{gradient} + \text{velocity term} \quad \leftarrow \text{New rule after adding velocity}$$

The velocity term equals the weighted average of the past gradients.

- *Adam*
Adam stands for ***adaptive moment estimation***.
- Adam keeps an exponentially **decaying average of past gradients, similar to momentum**.
- Whereas momentum can be seen as a ball rolling down a slope, Adam behaves like a heavy ball with ***friction*** to slow down the momentum and control it. Adam usually outperforms other optimizers because it helps train a neural network model much more quickly than the techniques we have seen earlier.
- `keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.0)`

- ***Number of epochs and early stopping criteria***

A training iteration, or *epoch*, is when the model goes a full cycle and sees the entire training dataset at once. The epoch hyperparameter is set to define how many iterations our network continues training. The more training iterations, the more our model learns the features of our training data. To diagnose whether your network needs more or fewer training epochs, keep your eyes on the training and validation error values.

Epoch 1, Training Error: 5.4353, Validation Error: 5.6394

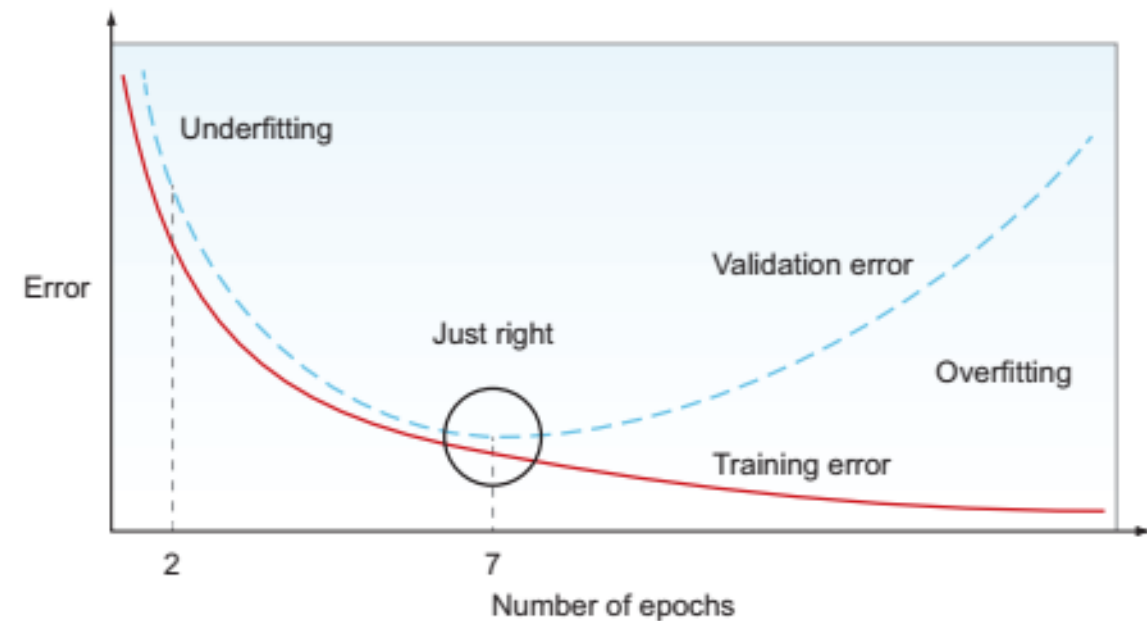
Epoch 2, Training Error: 5.1364, Validation Error: 5.2216

Epoch 3, Training Error: 4.7343, Validation Error: 4.8337

Epoch 6, Training Error: 3.7312, Validation Error: 3.8324

Epoch 7, Training Error: 3.5324, Validation Error: 3.7215

Epoch 8, Training Error: 3.7343, Validation Error: 3.8337



- **Early stopping**

Early stopping is an algorithm widely used to determine the right time to stop the training process before overfitting happens. It simply monitors the validation error value and stops the training when the value starts to increase. Here is the early stopping function in Keras:

- `EarlyStopping(monitor='val_loss', min_delta=0, patience=20)`
-

The `EarlyStopping` function takes the following arguments:

□ **monitor**—The metric you monitor during training. Usually we want to keep an eye on `val_loss` because it represents our internal testing of model performance. If the network is doing well on the validation data, it will probably do well on test data and production.

□ **min_delta**—The minimum change that qualifies as an improvement. There is no standard value for this variable. To decide the `min_delta` value, run a few epochs and see the change in error and validation accuracy. Define `min_delta` according to the rate of change. The default value of 0 works pretty well in many cases.

□ **patience**—This variable tells the algorithm how many epochs it should wait before stopping the training if the error does not improve. For example, if we set `patience` equal to 1, the training will stop at the epoch where the error increases. We must be a little flexible, though, because it is very common for the error to oscillate a little and continue improving. We can stop the training if it hasn't improved in the last 10 or 20 epochs.

Regularization techniques to avoid overfitting

- **L2 regularization**
- The basic idea of L2 regularization is that it penalizes (punish) the error function by adding a regularization term to it.
- This, in turn, reduces the weight values of the hidden units and makes them too small, very close to zero, to help simplify the model.
- $\text{error function}_{\text{new}} = \text{error function}_{\text{old}} + \text{regularization term}$

$$\text{L2 regularization term} = \frac{\lambda}{2m} \times \sum \|w\|^2$$

- where lambda (λ) is the regularization parameter, m is the number of instances, and w is the weight. The updated error function looks like this:

$$\text{error function}_{\text{new}} = \text{error function}_{\text{old}} + \frac{\lambda}{2m} \times \sum \|w\|^2$$

- Why does L2 regularization reduce overfitting?

the optimizer calculates the derivative of the error, multiplies it by the learning rate, and subtracts this value from the old weight. Here is the backpropagation equation that updates the weights:

Since we add the regularization term to the error function, the new error becomes larger than the old error.

This means its derivative ($\partial \text{Error} / \partial W_x$) is also bigger, leading to a smaller W_{new} .

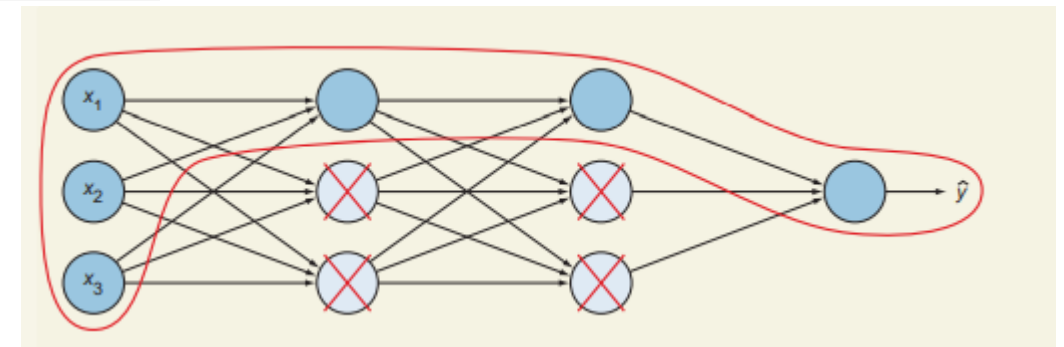
L2 regularization is also known as weight decay, as it forces the weights to decay toward zero (but not exactly zero).

In practice, L2 regularization does not make the weights equal to zero. It just makes them smaller to reduce their effect. A large regularization parameter (λ) lead to negligible weights. When the weights are negligible, the model will not learn much from these units. This will make the network simpler and thus reduce overfitting

$$W_{\text{new}} = W_{\text{old}} - \alpha \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

Labels in the diagram:
 - Old weight points to W_{old}
 - Derivative of error with respect to weight points to $\frac{\partial \text{Error}}{\partial W_x}$
 - New weight points to W_{new}
 - Learning rate points to α

$$\text{error function}_{\text{new}} = \text{error function}_{\text{old}} + \frac{\lambda}{2m} \times \sum \|w\|^2$$

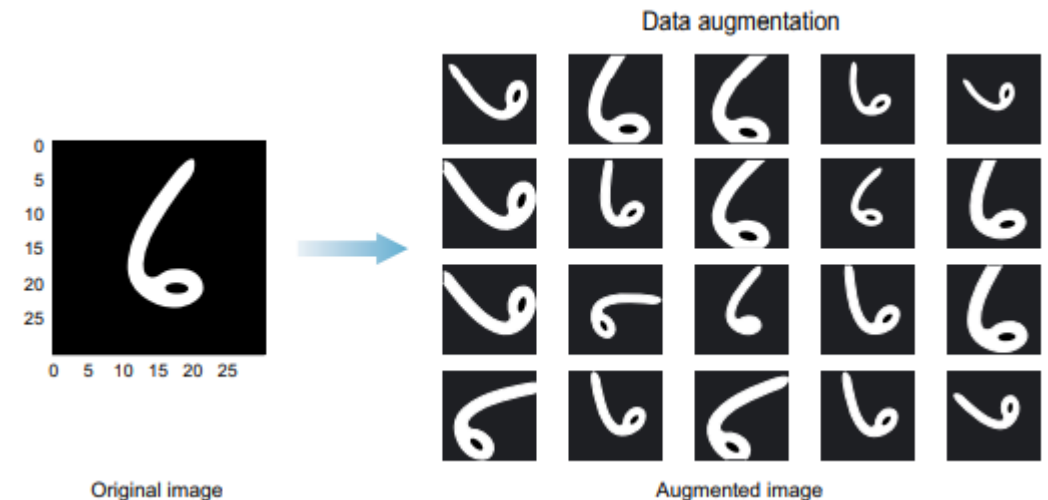


```
model.add(Dense(units=16, kernel_regularizer=regularizers.l2( $\lambda$ ), activation='relu'))
```

- **Dropout layers**
- Dropout is another regularization technique that is very effective for simplifying a neural network and avoiding overfitting.
- The dropout algorithm is fairly simple: at every training iteration, every neuron has a probability p of being temporarily ignored (dropped out) during this training iteration. This means it may be active during subsequent iterations. While it is counterintuitive to intentionally pause the learning on some of the network neurons, it is quite surprising how well this technique works.
- The probability p is a hyperparameter that is called dropout rate and is typically set in the range of 0.3 to 0.5. Start with 0.3, and if you see signs of overfitting, increase the rate.
- Both L2 regularization and dropout aim to reduce network complexity by reducing its neurons' effectiveness.
- The difference is that dropout completely cancels the effect of some neurons with every iteration.
- while L2 regularization just reduces the weight values to reduce the neurons' effectiveness. Both lead to a more robust, resilient neural network and reduce overfitting. It is recommended that you use both types of regularization techniques in your network.

- **Data augmentation**
- we can augment our training data by generating new instances of the same images with some transformations.
- Data augmentation can be an inexpensive way to give your learning algorithm more training data and therefore reduce overfitting.

The many image-augmentation techniques include flipping, rotation, scaling, zooming, lighting conditions, and many other transformations that you can apply to your dataset to provide a variety of images to train on.



```
from keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True)
datagen.fit(training_set)
```